Retainable Interactive Computation: Operational Semantics, Algebraic Effects, and Crash-Consistent Recovery in the Agentic Lambda Abstraction Network (ALAN)

ALAN Research Monograph

October 2026

Abstract

Contemporary frameworks for autonomous and language-model-driven agents are overwhelmingly engineered as ad hoc orchestration libraries: they embed control flow inside opaque host-language closures, conflate probabilistic model completions with authoritative state transitions, and rely on the incidental survival of an operating-system process to carry residual computation forward. When a process fails mid-execution, or when a higher-order coordinator must supervise, bound, retry, or cancel a subordinate agent, the computation itself possesses no independent mathematical existence.

This monograph develops the theoretical foundations of **ALAN (Agentic Lambda Abstraction Network)**, a programming-language and computation-model architecture whose central thesis is that *agentic computation must be a first-class, retainable mathematical object*. Rather than treating agents as black-box callbacks connected by workflow arrows, ALAN formulates an agent as a version-bound, finite-step interactive transition system whose residual control, typed environment, correlation obligations, and pending effect intents are explicitly reified as serializable projections.

We situate ALAN at the intersection of five classical traditions in theoretical computer science and distributed systems: (I) Church, Turing, and Felleisen's theory of computability versus *macro-expressiveness* [4, 15, 5]; (II) Plotkin and Turi's *structural* and *mathematical operational semantics* [11, 14]; (III) Milner, Hewitt, and Agha's calculi of *mobile processes and actor configurations* [6, 8, 1]; (IV) Moggi, Plotkin, Pretnar, and Xia et al.'s *algebraic effects and coinductive Interaction Trees* [10, 12, 16]; and (V) Schneider, Lamport, and Chandy's theory of *fault-tolerant state machines, causal ordering, and consistent cuts* [7, 3, 13].

Synthesizing these five pillars, we present four groups of definitions, arguments and implementation evidence, with general congruence and dependent recovery simulations retained as open obligations: (1) the relative macro-expressiveness of supervision, coordination, and bounded snapshot-cursor iteration (Λ_{ALAN}); (2) a syntactic bimorphism lowering high-level interactions into standard Interaction Trees over a split-phase signature $E_{\mathrm{split}} = \mathsf{Emit} \uplus \mathsf{Receive}$, paired with a novel *Split-Phase Journaled Handler Algebra* ($\mathcal{H}_{\mathrm{journal}}$) whose per-intent state evolves monotonically over a four-stage consistent epistemic poset $\mathbb{O}_{\mathrm{eff}}^{\mathrm{cons}} = \{\bot_{\mathrm{unauth}}, \mathsf{Suppressed}(r), \mathsf{Unknown}, \mathsf{Known}(v)\}$ completed by an improper fault element $\top_{\mathrm{conflict}}$ into a five-kind bounded lattice $\widehat{\mathbb{O}}_{\mathrm{eff}}$; (3) the orthogonality of provenance, communication, and lifecycle authority graphs ($\mathcal{G}_{\mathrm{prov}} \perp \mathcal{G}_{\mathrm{comm}} \perp \mathcal{G}_{\mathrm{life}}$) and a proposed, unproved general congruence claim for budget-indexed, interface-scoped failure/divergence bisimulation ($\approx^{\Delta}_{I,\mathcal{B}}$) preserving dual mailbox-admission and continuation-consumption refusals; and (4) an FLP-style impossibility theorem for unconditional crash bisimulation across unreconciled non-idempotent in-flight effects, leading to the *Two-Regime Fundamental Theorem of Durable Interactive Recovery*, which separates retained-state and conservative-uncertainty safety from proposed symmetric and directed future simulations. Peer interference, resource/closure stability, endpoint coupling and scheduling require additional arguments; retry expenditure and external effects cannot be hidden to restore equivalence.

Contents

1 Introduction: The Ontological Crisis of Agentic Computation **3**
- 1.1 Orchestration Frameworks versus Models of Computation . . . 3
- 1.2 Durable Workflows versus Durable Computation . . . 3
- 1.3 Distinguishing Engineering Synthesis from Mathematical Contributions . . . 4

2 Pillar I: Computational Universality, Bounded Realizability, and Felleisen Expressiveness **5**
- 2.1 Beyond the Turing Tarpit . . . 5
- 2.2 Macro-Eliminability of Supervision and the Phase 1C Falsification . . . 6
- 2.3 Bounded Data-Dependent Iteration and Snapshot Cursors (Phase 1G) . . . 6

3 Pillar II: Structural and Mathematical Operational Semantics **8**
- 3.1 Small-Step Operational Semantics of Pure Local Reduction (δ_v) . . . 8
- 3.2 Global Transition Rules at the Runtime Boundary . . . 9

4 Pillar III: Dynamic Concurrency, Orthogonal Authority, and Compositional Congruence **9**
- 4.1 The Four-Tier Identity Hierarchy and Orthogonal Authority Graphs . . . 9
- 4.2 Two-Tier Injective Identity Algebra . . . 10
- 4.3 Bounded Template Invocations and Conditional Congruence . . . 10

5 Pillar IV: Algebraic Effects, Split-Phase Operational Handlers, and the Epistemic Effect Lattice **13**
- 5.1 Split-Phase ITree Lowering versus the Asynchronous Handler Algebra . . . 13
- 5.2 Separating Probabilistic Intelligence from Deterministic Execution . . . 13

6 Pillar V: Crash-Consistent Operational Semantics and the Two-Regime Fundamental Theorem **14**
- 6.1 Two-Tier Configurations, the Recovery Operator, and the Window-W_3 Impossibility Theorem . . . 14
- 6.2 The Two-Regime Fundamental Theorem of Durable Computation . . . 14

7 Conclusion **15**

1 Introduction: The Ontological Crisis of Agentic Computation

1.1 Orchestration Frameworks versus Models of Computation

The rapid proliferation of large language models (LLMs) has catalyzed an industry of software frameworks purporting to build "autonomous agents." Viewed through the lens of programming-language theory, however, nearly all contemporary agent frameworks are variations on a single, fragile idiom: *imperative host-language orchestration*. In such systems, an "agent" is a Python, TypeScript, or Go closure that loops over a mutable prompt buffer, invokes an external HTTP endpoint to sample tokens from a probabilistic model, parses tool-call strings from the completion, and executes arbitrary host callbacks in-place.

This architecture suffers from a foundational ontological confusion by conflating three layers that must be kept mathematically distinct:

1. **Probabilistic Epistemic Proposals:** A language model completion is not a computation step; it is an untrusted, nondeterministic external observation sampled from a stochastic oracle.

2. **Deterministic Residual Control:** The logic that validates an observation against a schema, updates local state, enforces budget ceilings, and selects the next branch of work is a pure, finite state transition.

3. **Authoritative World Transitions:** Admitting a message to a peer's inbox, mutating an external database, or incrementing the execution generation of a failed child computation are irreversible distributed commitments subject to partial failure, message reordering, and process death.

When a conventional agent framework embeds all three layers inside the call stack of a live OS process, the *continuation* of the agent—what remains to be done, which obligations are outstanding, and what authority the agent holds—is implicitly encoded in ephemeral instruction pointers, heap pointers, and open TCP sockets. If the executor process receives `SIGKILL` after dispatching an external tool request but before recording its receipt, the residual computation vanishes.

1.2 Durable Workflows versus Durable Computation

Even industrial "durable execution" engines (such as Temporal) address process crashes via event-sourced replay of host-language coroutines rather than reifying the computation itself. A decisive gulf separates *durable workflow bookkeeping* from *durable computation*:

- **First-Class Projections vs. Opaque Coroutines:** Workflow engines treat the program as an opaque host closure whose state is invisible except by re-executing host code from step 0. ALAN represents every computation as an explicit, version-bound instruction array P_v and serializable projection $\sigma = \langle pc, \rho, status, \kappa \rangle$.

- **Ordinary Supervision vs. Privileged Runtime Strata:** In workflow and actor engines, supervision and retry policies are privileged runtime mechanisms. In ALAN, a supervisor or coordinator is an ordinary program in the same calculus that consumes typed terminal observations and emits lifecycle intents through the same interpreter (`executor.Reduce`).

- **Split-Phase Effect Authority:** Rather than collapsing external activities into `Started/Completed`, ALAN factors every interaction across five distinct commit boundaries (Prepare → Commit → Authorize → Apply → Settle) with explicit epistemic outcomes (Suppressed vs. Unknown).

Foundational Work	Fundamental Contribution	Connection to ALAN
Church (1936) [4]; Turing (1936) [15]	λ-calculus and universal mechanical computation	Small, universal compositional core without privileged agent classes
Felleisen (1991) [5]	Rigorous distinction between computability and macro-expressiveness	Proving supervision is derived while typed outcome admission is essential
Plotkin (1981) [11]	Structural Operational Semantics (SOS)	ALAN's finite interpreter (`Reduce`) and global ledger transition rules
Turi & Plotkin (1997) [14]	Mathematical operational semantics & bialgebraic congruence	Compositional equivalence for nested agent networks
Milner, Parrow & Walker (1992) [8]; Hewitt (1973) [6]	Mobile processes (π-calculus) and actor configurations	Dynamic template invocation (`compose`), capability scoping, and identity
Moggi (1991) [10]; Plotkin & Pretnar (2009/2013) [12]	Computational monads and algebraic effect handlers	Separating pure reduction from durable intents and runtime adapter execution
Xia et al. (2020) [16]	Coinductive Interaction Trees (ITree)	Mathematical analogue to ALAN's retained interactive computations
Lamport (1978) [7]; Chandy & Lamport (1985) [3]	Causal happens-before ordering and consistent global cuts	Per-run observation serialization, generation routing, and replay cuts
Schneider (1990) [13]	Fault-tolerant deterministic state machines	Journaled replay and crash recovery extended to external uncertainty

Table 1: The intellectual lineage of ALAN across programming languages and distributed systems.

1.3 Distinguishing Engineering Synthesis from Mathematical Contributions

To situate this work precisely for a programming-languages audience, we distinguish what ALAN synthesizes from existing literature from where existing formalisms break down:

1. **Engineering Synthesis of Known Theory:** Representing local step execution (`Reduce`) as a deterministic transition function δ_v instantiates Plotkin's SOS [11]; lowering synchronous requests $\mathsf{Vis}(e, k)$ into split-phase Emit and $\mathsf{Receive}$ instructions is a standard free-monad homomorphism $\mathcal{S} : \mathsf{ITree}_{E,R} \to \mathsf{ITree}_{E_{\text{split}},R}$ [16]; and re-evaluating history rows under monotonically increasing epoch numbers $e \mapsto e + 1$ synthesizes Schneider's state-machine replay [13] and Burrows's fencing tokens [2].

2. **Specific Mathematical Contributions of This Work:**

   - *The Split-Phase Journaled Handler Algebra ($\mathcal{H}_{journal}$) and Consistent Epistemic Poset ($\mathbb{O}_{eff}^{cons}$):* Standard ITree interpretation (`interp`) applies a monadic handler $h : E_{\text{split}} \rightsquigarrow \mathsf{StateT}(\mathcal{D}, M)$ atomically inside the evaluation thread, which cannot model an effect in flight across a process crash or concurrent `CancelRun`. We formulate the asynchronous handler protocol over the four-stage consistent epistemic poset $\mathbb{O}_{\text{eff}}^{\text{cons}}$ completed by $\top_{\text{conflict}}$ into $\widehat{\mathbb{O}}_{\text{eff}}$ (where $\mathsf{Suppressed}(r)$ and $\mathsf{Known}(v)$ are strictly incomparable consistent elements whose join is $\top_{\text{conflict}}$) and prove the Cancellation–Authorization Dichotomy (Theorem 5.1).
   - *Relative Macro-Expressiveness of Supervision:* Using Felleisen's $\preceq_{\text{macro}}$ [5], we prove

both positive macro-eliminability of supervision and its exact negative boundary when typed terminal inbox admission is omitted (Theorem 2.3).

- *Inductive Congruence under Dual Refusals and Shared External Effects:* We show why standard weak bisimulation fails under hiding in the presence of asynchronous mailbox admission vs. synchronous continuation consumption, shared external endpoints $\mathcal{W}_\ell$, and shared root conservation envelopes $\mathcal{B}(r)$, and formulate an open conditional-congruence claim over $\mathcal{C}_{\mathrm{adm}}$ (Theorem 4.5), with interference, invocation and scheduler obligations still to be discharged.
- *Impossibility of Unconditional Crash Bisimulation and the Two-Regime Recovery Theorem:* We prove that no deterministic recovery operator without out-of-band world introspection can achieve symmetric forward bisimulation across an unreconciled non-idempotent Window-W_3 crash while guaranteeing at-most-once world mutation (Proposition 6.1), and distinguish conservative recovery safety from the stronger symmetric and directed simulation obligations (Theorem 6.2). The general peer/interference and reconciliation arguments remain open.

2 Pillar I: Computational Universality, Bounded Realizability, and Felleisen Expressiveness

2.1 Beyond the Turing Tarpit

Church [4], Turing [15], and Minsky's Theorem Ia [9] establish that two unbounded integer registers with increment-and-jump and decrement-and-branch-on-zero suffice to compute any partial recursive function. In an open, durable agent system, however, appealing to Turing completeness obscures two critical realities:

1. **Bounded Realizability:** Practical durable deployments enforce explicit resource ceilings $\mathcal{L} = (\mathit{MaxProgramBytes}, \mathit{MaxDataBytes}, \mathit{MaxObservationBytes}, \mathit{MaxOutputBytes})$ (each 1 MiB by default in `internal/executor/program.go`) and checked 64-bit integer arithmetic (`int64` overflow trapping to `ErrLimit`). Unbounded host callbacks violate finite-step scheduling.

2. **Computability vs. Macro-Expressiveness:** Following Felleisen [5], whole-program simulation via Gödel numbering is vacuous for language design because it destroys concurrency, local state isolation, and observational equivalence in contexts.

Definition 2.1 (Contextual Equivalence and Macro-Eliminability, after Felleisen [5])**.** Let $\mathcal{L} = (\mathsf{Syn}_{\mathcal{L}}, \mathsf{Conf}_{\mathcal{L}}, \longrightarrow_{\mathcal{L}}, \mathsf{Obs}_{\mathcal{L}})$ be an interactive calculus. Two terms $P, Q \in \mathsf{Syn}_{\mathcal{L}}$ are *contextually equivalent*, written $P \simeq_{\mathcal{L}} Q$, if $\mathsf{Obs}_{\mathcal{L}}(C[P]) = \mathsf{Obs}_{\mathcal{L}}(C[Q])$ for every closing context $C[\cdot]$. An extension $\mathcal{L}^+ = \mathcal{L} \cup \{F(\cdot_1, \ldots, \cdot_k)\}$ is *macro-expressible* in $\mathcal{L}$ ($\mathcal{L}^+ \preceq_{\mathrm{macro}} \mathcal{L}$) if there exists a homomorphic translation $\mathcal{T} : \mathsf{Syn}_{\mathcal{L}^+} \to \mathsf{Syn}_{\mathcal{L}}$ fixing $\mathcal{L}$-primitives such that $\mathcal{T}(F(t_1, \ldots, t_k)) = \Phi_F[\mathcal{T}(t_1), \ldots, \mathcal{T}(t_k)]$ for a fixed $\mathcal{L}$-context Φ_F and $\mathsf{Obs}_{\mathcal{L}^+}(P) = \mathsf{Obs}_{\mathcal{L}}(\mathcal{T}(P))$ for all closed P.

Theorem 2.2 (Inexpressibility of `Conditional` in Trace-Fixed Total `OneShot` Calculi)**.** *Let $\mathcal{L}_{\mathit{fixed}}$ be generated by atomic `OneShot` leaves L_i and sequential composition $P; Q$, where each leaf L_i is total, terminates normally without input-dependent failure or early exit, and emits a fixed nonempty sequence of observable effect labels $\vec{\alpha}(L_i) \in \mathsf{Act}^+$ independent of input data ρ. Let $\mathcal{L}^+_{\mathit{fixed}}$ adjoin $\mathsf{Conditional}(p, A, B)$ for a non-constant predicate $p(\rho) \in \{\mathsf{true}, \mathsf{false}\}$. Then $\mathcal{L}_{\mathit{fixed}} \prec_{\mathit{macro}} \mathcal{L}^+_{\mathit{fixed}}$.*

Proof. By structural induction on $M \in \mathsf{Syn}_{\mathcal{L}_{\mathrm{fixed}}}$, totality and normal termination of every leaf L_i ensure that in $P; Q$, P always emits its complete fixed trace $\vec{u}$ and passes control to Q, which

emits its complete fixed trace $\vec{v}$, making the concatenated trace $\pi(M, \rho) = \vec{u} \cdot \vec{v}$ independent of ρ. By contrast, $\mathsf{Conditional}(p, L_{\text{left}}, L_{\text{right}})$ with $\vec{\alpha}(L_{\text{left}}) \neq \vec{\alpha}(L_{\text{right}})$ emits input-dependent traces. (When leaves instead evaluate a pure reducer δ_v that selects residual control, `Conditional` is locally macro-eliminable via `branch` and `jump`.) □

2.2 Macro-Eliminability of Supervision and the Phase 1C Falsification

Theorem 2.3 (Macro-Eliminability of Bounded Supervision)**.** *In ALAN's core instruction calculus $\mathcal{L}_{ALAN}$ (`set`, `add`, `branch`, `jump`, `receive`, `emit`, `end`), a bounded supervisor $\mathsf{Supervise}(a_{child}, B_{retry}, obl, P_{init}, P_{succ})$ is locally macro-expressible without any privileged supervisor opcode or runtime role dispatch, if and only if the runtime boundary admits typed terminal observations (kind = "`terminal`", prov = "`runtime-terminal`") and correlated lifecycle intents/settlements (kind = "`lifecycle`").*

Proof. The constructive macro-expansion $\Phi_{\mathsf{Supervise}}$ is given by the 15-instruction inner supervisor in `internal/executor/examples.go` (`NestedProvision`, lines 52–69):

```
pc 0: receive(kind="message", prov="ordinary-message", src=Outer, gen=1, obl="inner-start") ->
    [started]
pc 1: emit(kind="message", target=Worker, gen=Ref("expected"), obl="worker-work", payload=
    P_init)
pc 2: receive(kind="terminal", prov="runtime-terminal", src=Worker, gen=Ref("expected"), obl=
    obl) -> [outcome]
pc 3: branch(Ref("outcome") == "succeeded", target=13, otherwise=4)
pc 4: branch(Ref("outcome") == "failed", target=5, otherwise=14)
pc 5: branch(Ref("spent") == Ref("limit"), target=14, otherwise=6)
pc 6: emit(dst="pending", kind="lifecycle", target=Worker, gen=Ref("expected"), obl=obl,
          payload={"operation": "Retry", "revision": Ref("revision")},
          updates={"spent": Offset("spent", +1)})
pc 7: receive(kind="lifecycle", prov="runtime-lifecycle", src=Worker, gen=Ref("expected"),
             obl=obl, op=Ref("pending")) -> [accepted, reason, successor, disposition]
pc 8: branch(Ref("accepted") == true, target=9, otherwise=14)
pc 9: set(dst="expected", A=Ref("successor"))
pc 10: set(dst="pending", A="")
pc 11: emit(kind="message", target=Worker, gen=Ref("expected"), obl="worker-work", payload=
    P_succ)
pc 12: jump(target=2)
pc 13: end(outcome="succeeded")
pc 14: end(outcome="failed")
```

At `pc = 6`, `emit` atomically commits the `Retry` intent, stores its canonical ID into `pending`, and increments `spent` by +1. Conversely, in the initial Phase 1 model where `DeliverTerminal` updated an external ledger without inserting a typed `"terminal"` inbox observation (Phase 1C regression in `docs/foundational/05-decisions.md`), no ordinary instruction at `pc = 2` could observe child failure, proving that typed outcome admission is an essential expressive boundary. □

2.3 Bounded Data-Dependent Iteration and Snapshot Cursors (Phase 1G)

Theorem 2.4 (Laminar Cursor Conservation and Strong Termination)**.** *Let P be any program passing `validateProgram(P)` under `CollectionSemanticVersion` (`internal/executor/collections.go`), with code length $L = |P.\texttt{Code}|$, maximum loop-snapshot capacity $M = \max_c c.\texttt{MaxItems}$, where c ranges over the `Collection` schemas of `foreach` instructions (set $M = 0$ if there are no loops), and actual finite loop nesting depth d. The public authoring compiler additionally enforces $d \leq$ `author.MaxNesting`; the raw program validator does not impose that cap. For any valid open projection σ_0:*

1. ***Snapshot Invariance:*** *Mutations to $\rho[\texttt{items}]$ inside a loop body leave the active cursor's cloned $\kappa[pc_{start}]$.Snapshot invariant.*

2. ***Cursor Well-Bracketedness:*** *At every reachable open projection σ, $\kappa[pc_{start}]$ exists iff $pc_{start} < \sigma.pc < P.\texttt{Code}[pc_{start}].\texttt{Otherwise}$. Executing* `end` *clears all cursors, even when its program counter remains inside a loop interval.*

3. ***Strong Termination Bound:*** *Every sequence of successful local reductions has length at most $L \cdot (M+2)^d$. A maximal sequence ends with terminal status, a blocked* `receive`*, or a rejected reduction. Excluding rejection additionally requires that all executed operands, schemas, resource checks, and successor program counters are valid;* `validateProgram(P)` *alone does not establish this assumption. The bound counts successful instruction reductions, not unsuccessful receive polling or elapsed external waiting.*

Proof. (1) `foreach` stores a deep clone of the evaluated source list in its cursor. Subsequent `next` reductions read only this snapshot, clone it forward, and increment its index; body mutations to the source variable do not change it.

(2) `validateCollections` enforces laminar `foreach`–`loop` intervals, a matching `next` immediately after each header, and a matching `loop` immediately before its exit. It forbids backward `jump`/`branch` targets and entry from outside into a loop body. Thus a newly entered body has a cursor created by its `foreach`; an initially active loop has one by `validateCursors`. On an outward forward exit, `cleanCursors` deletes every exited cursor. It also clears all cursors on `end`, so the interval equivalence is asserted only for open projections.

(3) On a stretch of forward instructions, the natural-valued distance $L - \sigma.pc$ strictly decreases. The former coordinate $\sigma.pc$ had the wrong sign. Replacing it in the active-cursor tuple does not give a ranking that decreases at every instruction: `loop` jumps backward without changing any cursor index, and the following `next` increments it in a separate reduction. In the five-instruction program `foreach; next; set; loop; end`, with a one-element snapshot already bound, `set` at $2 \to 3$ increases the old tuple $(0, 2) \to (0, 3)$, while `loop` at $3 \to 1$ increases the sign-corrected tuple $(0, L-3) = (0, 2) \to (0, L-1) = (0, 4)$. Loop entry and exit also change the active tuple's dimension. We therefore use a structural instruction-count argument rather than assert a global per-step lexicographic rank.

For one activation with immutable snapshot length $n \leq M$, `next` binds at most n elements and performs at most one final exhaustion check; `loop` executes at most once per bound element. Induct on laminar loop depth: at depth zero all nonterminal steps advance the program counter, so each instruction executes at most once. At greater depth, the induction hypothesis makes each strictly nested loop activation finite; after collapsing those activations, each body pass contains only forward control flow up to the enclosing `loop` or an outward exit. Hence there are at most M body passes and $M + 1$ executions of `next` per activation. Outward branches and early `end` only shorten the traversal.

For an initially active loop, starting at `next` leaves at most $n - \mathit{Index}$ bindings. Starting inside its body requires $\mathit{Index} \geq 1$ by `validateCursors`, so the current partial pass plus the remaining passes number at most $1 + n - \mathit{Index} \leq n$. Each nested header is encountered at most once per enclosing body pass. Applying the bounds from outermost to innermost gives at most $(M+1)^d$ executions of any fixed instruction, including exhaustion checks. Summing over the L instructions yields at most $L(M+1)^d \leq L(M+2)^d$ successful reductions. Failed checks and waiting without an eligible observation add none. This proves the bound without claiming successful completion or a bound on external waiting. □

Bounded adaptive application (Phase 1H). `examples/adaptive` uses the existing public authoring operations to accept seed work after initialization and derive each ordered successor frontier from the preceding round's returned outcomes. A single retained three-token `ForEach` controls at most three rounds, with at most eight application-accepted leaf items across the obligation. Each round emits its item invocations, consumes all required outcomes through their original operation handles, checks the complete candidate frontier for identity collisions

and remaining capacity, and retains a one-row decision before emitting successor work. Application work IDs are distinct from runtime operation identities. The full accepted-frontier count is retained before its first item intent; runtime invocation, created-run and external-attempt charges remain separate facts.

The scalar policy maps returned observations 0/1/2 to zero/one/two ordered children, using the finite integer identity scheme in the application contract. The historical paired controlled cases used the same request and retained artifacts: changing only gamma's returned observation from 2 to 0 changed frontiers `[1,2,3] / [7,10,11] / [31]` into `[1,2,3] / [7]`. Direct and wrapped deployment use the identical adaptive artifact at maximum leaf depths 2 and 3. Additional rounds do not increase invocation depth. Request round/work limits persist under the shared root envelope of 32 accepted invocations, 64 created runs and 96 external reservations, with physical capacity 1 or 2; the fixed root run is separate provisioning cost.

Protocol success does not mean exhaustive application completion. Work- or round-limit stops return `complete=false` with the entire unperformed frontier; only exhaustion returns `complete=true`. A required unknown outcome blocks the round barrier and remains spent, even if a sibling has a known failure. Neither an unknown outcome nor an absent response is converted into observation zero or permission for replacement work. Root closure may prevent any application reply. The effectful profile directly reuses the existing two-member evidence helper, accepts questions of at most 512 UTF-8 bytes, and consumes only its returned disposition for branching; complete answers and citations remain inspectable in the original retained outcomes. It establishes no answer-quality or semantic-completeness claim.

These are bounded implementation results, not a general expressiveness or recovery proof. The source-bound Task2 report records passing controlled direct/nested and paired-result cases and byte/profile checks at manifest `fd7ab336...`; those results predate the rebase onto `56892cb...` and do not qualify the rebased source. Process recovery, unknown/closure schedules, isolated replay, independent faulty-application rejection, final suites and review are qualified separately by source-tagged records under `docs/foundational/phase1h`; only records matching the relevant source establish that source's tested status. Replay checks retained protocol consistency; the independent expected-work ledger checks whether the application follows the intended frontier policy.

3 Pillar II: Structural and Mathematical Operational Semantics

3.1 Small-Step Operational Semantics of Pure Local Reduction (δ_v)

Let $x = (a, g) \in \mathsf{RunID}$ be a logical execution identity, $n \in \mathbb{N}^+$ the next commit sequence, and $\sigma = \langle pc, \rho, status, \kappa \rangle$ an open projection ($status =$ "`open`"). The pure single-step reducer δ_v (`executor.Reduce` in `internal/executor/program.go`) defines the judgment $P, x, n, \mathcal{L} \vdash \langle \sigma, o_{\text{opt}} \rangle \xrightarrow{\delta_v} \langle \sigma', \vec{\iota} \rangle$:

$$\textbf{(R-Receive)} \quad \frac{\begin{array}{c} I_{pc} = \mathsf{Receive}(\mathcal{M}, \beta) \quad o_{\text{opt}} = \mathsf{Some}(o) \quad o.Run = x \quad \|o\|_{\text{json}} \le \mathcal{L}.MaxObs \\ \langle o.Kind, o.Prov, o.Src, o.SrcGen, o.Obl, o.Op \rangle = \\ \langle \mathcal{M}.kind, \mathcal{M}.prov, [\![\mathcal{M}.e_{\text{src}}]\!]_\rho, [\![\mathcal{M}.e_{\text{gen}}]\!]_\rho, [\![\mathcal{M}.e_{\text{obl}}]\!]_\rho, [\![\mathcal{M}.e_{\text{op}}]\!]_\rho \rangle \\ \mathsf{schemaValid}_P(pc, o.Payload) \quad \rho' = \mathsf{bindPayload}(\rho, \beta, o.Payload) \end{array}}{P, x, n, \mathcal{L} \vdash \langle \sigma, \mathsf{Some}(o) \rangle \xrightarrow{\delta_v} \langle \mathsf{clean}_P(\langle pc + 1, \rho', \text{"}\mathtt{open}\text{"}, \kappa \rangle), [] \rangle}$$

$$\textbf{(R-Emit)}\quad \frac{\begin{array}{c} I_{pc} = \mathsf{Emit}(dst_{\text{opt}}, \mathcal{E}, \Upsilon) \quad o_{\text{opt}} = \mathsf{None} \quad id = \texttt{json}([a, g, n, 0]) \\ \iota = \langle ID : id,\ Run : x,\ Seq : n,\ Ord : 0,\ Kind : \mathcal{E}.kind, \\ Target : [\![\mathcal{E}.e_{\text{target}}]\!]_\rho,\ Gen : [\![\mathcal{E}.e_{\text{gen}}]\!]_\rho,\ Obl : [\![\mathcal{E}.e_{\text{obl}}]\!]_\rho,\ Payload : [\![\Pi]\!]_\rho \rangle \\ \mathsf{authorIntentValid}_P(\iota) \quad \rho_1 = \rho[\forall (k \mapsto e_u) \in \Upsilon : k \mapsto [\![e_u]\!]_\rho] \\ \rho_2 = \rho_1[dst_{\text{opt}} \mapsto \mathsf{Str}(id)] \quad \|[\iota]\|_{\text{json}} \le \mathcal{L}.MaxOut \end{array}}{P, x, n, \mathcal{L} \vdash \langle \sigma, \mathsf{None} \rangle \xrightarrow{\delta_v} \langle \mathsf{clean}_P(\langle pc + 1, \rho_2, \text{"}\mathsf{open}\text{"}, \kappa \rangle), [\iota] \rangle}$$

Lemma 3.1 (Purity and Strong Determinism of δ_v)**.** *For any valid program P, run identity $x = (a, g)$, sequence $n \in \mathbb{N}^+$, limits $\mathcal{L}$, open projection σ, and optional observation o_{opt}, $\xrightarrow{\delta_v}$ is a partial function producing unique canonical JSON bytes for σ' and $\vec{\iota}$.*

3.2 Global Transition Rules at the Runtime Boundary

Embedding δ_v into a global configuration Γ yields the atomic commit and authorization transitions:

$$\textbf{(G-Commit)}\quad \frac{\begin{array}{c} p = \langle x = (a, g), e, n + 1, o_{\text{opt}}, \sigma', \vec{\iota} \rangle \in \Gamma.\mathsf{Prop} \quad \Gamma.\mathsf{Subj}[a] = \langle g_{\text{curr}}, owner, w, retired \rangle \\ g = g_{\text{curr}} \quad \neg retired \quad \Gamma.\mathsf{Runs}[x] = \langle P_v, e_{\text{curr}}, n_{\text{curr}}, \sigma \rangle \\ e = e_{\text{curr}} \quad n = n_{\text{curr}} \quad \sigma.status = \text{"}\mathsf{open}\text{"} \\ (o_{\text{opt}} = \mathsf{Some}(o) \implies o \in \Gamma.\mathsf{Obs}[x] \wedge o.consumed = \bot) \quad \neg\mathsf{scopeClosed}(\Gamma, x) \end{array}}{\Gamma \xrightarrow{\mathsf{Commit}(x, e, n+1)} \Gamma'}$$

$$\textbf{(G-Authorize)}\quad \frac{\begin{array}{c} \iota \in \Gamma.\mathsf{Int} \quad \iota.ID \notin \mathrm{dom}(\Gamma.\mathsf{Att}) \quad x = \iota.Run = (a, g) \\ \Gamma.\mathsf{Subj}[a] = \langle g_{\text{curr}}, owner, w_{\text{curr}}, retired \rangle \quad \Gamma.\mathsf{Runs}[x] = \langle P_v, e, n, \sigma \rangle \\ allowed \iff (g = g_{\text{curr}} \wedge \neg retired \wedge w_{\text{curr}} = \iota.revision \wedge \sigma.status = \text{"}\mathsf{open}\text{"} \\ \wedge \neg\mathsf{scopeClosed}(\Gamma, x) \wedge \mathsf{grant}(\Gamma.\mathsf{Perm}, \iota)) \end{array}}{\Gamma \xrightarrow{\mathsf{Authorize}(\iota.ID)} \Gamma\big[\mathsf{Att}[\iota.ID] \leftarrow \langle Allowed : allowed,\ Reason : reason \rangle\big]}$$

4 Pillar III: Dynamic Concurrency, Orthogonal Authority, and Compositional Congruence

4.1 The Four-Tier Identity Hierarchy and Orthogonal Authority Graphs

Unlike the π-calculus [8] or classical actor models [6, 1], ALAN separates identity into four tiers: (1) Retained Computation (P_v, σ); (2) Logical Subject $a \in \mathsf{Subj}$; (3) Logical Execution (Run) $x = (a, g) \in \mathsf{RunID}$; and (4) Physical Activation $(x, e) \in \mathsf{RunID} \times \mathbb{N}^+$, alongside ownership revision $w \in \mathbb{N}^+$.

Theorem 4.1 (Authority Non-Amplification and Graph Orthogonality — Invariant I3)**.** *In ALAN's operational semantics, the provenance graph $\mathcal{G}_{prov}$, communication/observation graph $\mathcal{G}_{comm}$, and lifecycle authority graph $\mathcal{G}_{life}$ are mutually orthogonal ($\mathcal{G}_{prov} \perp \mathcal{G}_{comm} \perp \mathcal{G}_{life}$): holding send or terminal-observation permissions in $\mathcal{G}_{comm}$ never enables an accepted lifecycle transition (`Retry`, `CancelRun`, `AbandonRun`, `RetireSubject`) in $\mathcal{G}_{life}$.*

4.2 Two-Tier Injective Identity Algebra

To eliminate receipt spoofing, transport domain shadowing, and cross-store operation key collisions, ALAN defines:

$$\mathsf{AdmKey}(d, \tau_{\text{raw}}) \triangleq \texttt{json}\big([\text{"}\texttt{admission}\text{"},\ d,\ \tau_{\text{raw}}]\big), \quad d \in \{\texttt{input}, \texttt{message}, \texttt{terminal}, \texttt{lifecycle}, \texttt{external}, \texttt{adapter}\}, \tag{1}$$

$$\mathsf{ExtKey}(\mathcal{N}_{256}, (a, g), n, k) \triangleq \texttt{json}\big([\text{"}\texttt{alan/external/v2}\text{"},\ \mathcal{N}_{256},\ a,\ g,\ n,\ k]\big), \quad \mathcal{N}_{256} \in \{0,1\}^{256}. \tag{2}$$

4.3 Bounded Template Invocations and Conditional Congruence

Theorem 4.2 (Root Resource Conservation and Waiting-Parent Slot Freedom)**.** *Let r be a composition root with envelope $\mathcal{B}(r) = \langle D_{\max}, I_{\max}, R_{\max}, E_{\max}, C_{phys} \rangle$ (`internal/executor/invocation.go`). Across arbitrary interleavings of local steps, template invocations, external adapter calls, root closures, and `SIGKILL` recoveries: (1) total created runs in r's tree never exceed $R_{\max}$, invocations never exceed $I_{\max}$, effect reservations never exceed $E_{\max}$, and depth never exceeds $D_{\max}$; and (2) a parent run suspended at `AwaitInvocation` occupies zero physical transport slots in C_{phys}.*

Definition 4.3 (Admissible Contexts, Dual Readiness, and Budget-Indexed Bisimulation $\approx^{\Delta}_{I,\mathcal{B}}$)**.** The grammar of admissible composition contexts $\mathcal{C}_{\text{adm}}(I_{\text{in}} \Rightarrow I_{\text{out}})$ is generated by:

$$C[\cdot] \ ::= \ [\cdot] \ \mid \ C[\cdot] \parallel_{\mathcal{I}} Q \ \mid \ Q \parallel_{\mathcal{I}} C[\cdot] \ \mid \ \mathsf{hide}\ H\ \mathsf{in}\ C[\cdot] \ \mid \ \mathsf{Invoke}_{T,r}(C[\cdot]), \qquad H \cap \mathcal{E}_{\text{ext}} = \emptyset.$$

Structural admissibility does not itself entitle a context to successful completion. Completion additionally requires contract-based accounting of prior spending, outstanding reservations, remaining candidate/peer/wrapper demand and depth at every reachable prefix. Insufficient capacity remains a valid refusal experiment.

Because mailbox admission (`AdmitMessage`/`DecideLifecycle`) is asynchronous with respect to a run's program counter while observation consumption (`Reduce` at `receive`) is synchronous with the 6-tuple guard $\mathcal{M}$, every configuration Γ exposes two refusal/readiness observables at interface I: *mailbox admission readiness* $\mathsf{AdmReady}_I(\Gamma) \subseteq \Sigma_{\text{in}}$ (which target subjects/generations are open vs. closed/retired) and *continuation consumption readiness* $\mathsf{WaitReady}_I(\Gamma) \subseteq \mathsf{Match}_I$ (which 6-tuple guards $\mathcal{M}$ are awaited by suspended `receive` instructions). External interactions are labeled by open transitions $\mathsf{extAuth}(r_{\text{root}}, g_{\text{root}}, \ell, \Pi, \Delta\mathcal{B}) \in \mathcal{E}^{\text{req}}_{\text{ext}}$ and environment-quantified responses $\mathsf{extSettle}(\iota.ID, v) \in \mathcal{E}^{\text{resp}}_{\text{ext}}$ $(\forall v \in \mathsf{Val})$.

A symmetric relation $\mathcal{R}$ is a *Budget-Indexed, Interface-Scoped Failure/Divergence Bisimulation* ($\approx^{\Delta}_{I,\mathcal{B}}$) if whenever $(\Gamma_1, \Gamma_2) \in \mathcal{R}$:

1. **Weak Transfer & Budget Preservation:** $\Gamma_1 \xrightarrow{\alpha}_H \Gamma_1'$ implies $\Gamma_2 \xRightarrow{\hat{\alpha}}_H \Gamma_2'$ with $(\Gamma_1', \Gamma_2') \in \mathcal{R}$ and equal root resource debits $\Delta\mathcal{B}(\Gamma_1 \to \Gamma_1') = \Delta\mathcal{B}(\Gamma_2 \Rightarrow \Gamma_2')$;
2. **Dual Refusal Preservation at Quiescence:** If Γ_1 is H-quiescent, $\Gamma_2 \xRightarrow{\epsilon}_H \Gamma_2'$ for an H-quiescent Γ_2' with $(\Gamma_1, \Gamma_2') \in \mathcal{R}$, $\Delta\mathcal{B} = \vec{0}$, $\mathsf{AdmReady}_I(\Gamma_1) = \mathsf{AdmReady}_I(\Gamma_2')$, and $\mathsf{WaitReady}_I(\Gamma_1) = \mathsf{WaitReady}_I(\Gamma_2')$;
3. **Divergence Sensitivity (Δ):** $\Gamma_1 \uparrow_H \iff \Gamma_2 \uparrow_H$.

Premise audit. Definition of $\approx^{\Delta}_{I,\mathcal{B}}$ requires symmetric weak transfer with equal debit on every matched step, zero-debit matching quiescence with dual readiness, and divergence sensitivity. It is stronger than schema compatibility or upper-bound contract refinement. Unequal-charge

local/helper implementations do not refute a theorem assuming it; equal final charges do not establish it. Parallel reasoning additionally needs relation stability under actual residual-budget, authority and closure interference.

Lemma 4.4 (Nonblocking Turn Structure; Confluence Obligation)**.** *For a finite workload while* `Store.Run` *continues without cancellation, operational-limit exhaustion or store error, its loop offers completion, internal delivery, external launch and ready-local-reduction opportunities. An unmatched receive does not itself hold a transport slot, and background transport does not hold the store mutex across network I/O.*

Code-level justification and limit. The ready-run scan rotates, but `selectReady` loads and scans observations; no constant-time skip or exact global turn bound is claimed. This supports nonblocking structure, not endpoint fairness or schedule equality. Local determinism and termination do not establish a diamond property for competing admissions, closure/authorization or shared endpoint updates. The former unique-quiescent-state inference is withdrawn; general confluence remains open. Interleaving semantics may preserve genuine nondeterminism.

Theorem 4.5 (Conditional Compositional Congruence; Open General Claim)**.** *The proposed implication is: for every context* $C[\cdot] \in \mathcal{C}_{adm}$, $\Gamma_A \approx^{\Delta}_{I_A,\mathcal{B}} \Gamma_{A'}$ *implies* $C[\Gamma_A] \approx^{\Delta}_{I_C,\mathcal{B}} C[\Gamma_{A'}]$*, under the originally stated hypotheses: (H1) private namespace separation, declared-port interaction and non-extrusion; (H2) root-correlated external provenance and equal matched debits; (H3) exact six-field correlation routing; (H4) nonblocking peer progress under explicit scheduling/environment assumptions; and (H5) a valid well-founded rank for relevant hidden interaction cycles.*

Status: unproved over the stated grammar. The label preserves the historical claim, not a certification. The former proof is retained in `docs/foundational/phase1i/evidence/manuscript/original-excerpts.md`. Missing arguments are:

1. **Parallel interference:** equal pre-step residual budgets do not prove that the component relation survives a peer debit, closure or authority change. Readiness, outstanding obligations and shared world-response coupling must also be preserved.
2. **Invocation:** admission/refusal, depth, member creation and charges require a separate induction case. A singleton wrapper costs one invocation, one created run and another depth; rejection creates no child/debit. It is not cost-free parallel/hiding syntax, and child/wrapper outcomes differ.
3. **Hiding/progress:** establish transfer, dual readiness and divergence for the actual hidden channels, without erasing shared charges, meaningful refusals, effects or unresolved obligations. The general loop-ranking obligation of Theorem 2.4 is not repaired here.
4. **Concrete scheduling:** abstract trace-set equality differs from a selected `Store.Run` trace. Pure steps, identity ordering and a volatile cursor lost on reopen may change the chosen interleaving. Fairness does not imply confluence or an identical contention winner.

Bounded behavioral contracts (Phase 1I). The frozen K1 contract separates schema compatibility, asymmetric local refinement and symmetric equivalence. Independently authored local and one-helper candidates preserve a request id and compute a Boolean complement, with explicit failure, missing-reply and pending modes. Five named templates (bare, sequential, parallel, hidden and one wrapper), finite payloads and residual allowances define 3,840 configurations per reference/candidate graph and 84 contract-fixed completion-entitled answer configurations. The reference $C[K1]$, local component relation and candidate products are checked separately. Artifact correspondence inspection, finite model checking, sampled runtime behavior and replay are distinct evidence categories.

Bounded negative result: root success fences a peer. An executed independent model extension reflecting `compositionScopeClosed` (`invocation.go:165--177`) exposes a missing assumption in the frozen parallel reference product. At request id 1/value 0, answer mode, invocation/run limits 3 and depth 2, a correct candidate result lets the root caller succeed before peer S admission; S is then refused with `scope_closed`. Initial completion entitlement is retained and no external `close_root` event occurs. This happens for K1 and both candidate graphs. It falsifies the frozen parallel completion premise/runtime correspondence, not an implication requiring a valid $C[K1]$ premise or the stronger exact-debit congruence premise. The source-specific boundary-extension census records all eight initially entitled parallel configurations failing and the other 76 completing in those finite graphs; the failures remain in scope. Earlier positive reference results used optimistic semantics omitting this ordinary-root-terminal guard. This is bounded model evidence; concrete runtime confirmation and final review are separately qualified.

Reference-model corrections preserve their original failures: collapsed helper/wrapper failure consumption and terminal freeze, and later premature payload visibility. Corrected graph results supersede those versions without rewriting their evidence. Preliminary source-specific checks live under `docs/foundational/phase1i/evidence/model`; they are neither a general theorem nor final runtime qualification. Weak-contract resource and equal-charge scheduling probes retain separate premises and traces. Phase1I qualification/review records govern the executed scope. Acyclic witnesses avoid the general loop-ranking obligation; pure own-state recovery does not establish effectful substitution. Replacing code beneath retained continuations is excluded.

Retained joining and report-source lifetime (Phase 1J). The separately frozen J1 experiment preserves A, B, S, the static root/peer topology, the peer's 32-step prefix, and the original resource allowances. Its root consumes the candidate outcome, retains it, and waits for a correlated ordinary report of the peer's actual S outcome before deciding either success or failure. The peer reports every known S outcome and then succeeds locally; this local status means reporting-program completion, not S success or guaranteed report delivery. An independent causal checker connects the report to the peer's original invocation handle and actually consumed observation. A valid-schema premature report can fool the trusting root while violating J1.

Within the eight original parallel answer configurations (two ids, two values, depth caps 2 or 3, invocation and created-run limits 3, zero effects), the independent finite products preserve the checked resource and root-lifetime obligations but admit a new completion counterexample for K1, A and B: S outcome consumption, report commitment, peer local Success, then first report authorization refused as `source_closed`. The report is an ordinary message, so the invocation-reply exemption does not apply; the root remains open at its required receive. Eventual service does not make a refused authorization successful. This is a bounded counterexample under the modeled local/service interleavings, not an observed failure of the unchanged `Store.Run` scheduler and not an impossibility theorem for every public join protocol.

Concrete delayed-peer samples using unchanged `Store.Run` at physical capacity one demonstrate the intended intermediate state: candidate outcome consumed, S still unadmitted, root open at its join receive. They complete with dynamic charges $A = (2, 2, 0)$, $B = (3, 3, 0)$, plus the two fixed runs, while the original C0 failure persists even with larger capacity. Four acknowledged process-kill/fresh-open samples retain the unfinished join or original committed report and complete. These sampled results and their adverse controls are separately qualified in `docs/foundational/phase1j/02-qualification.md`. They support the causal value of retaining the root in those executions; they do not establish all-schedule completion. The next exposed obligation concerns the source authority needed by required ordinary communication until authorization is retained. The general congruence, compiler and effectful-recovery obligations remain open. The structured-loop termination bound was corrected separately in

the preceding manuscript revision; J1's finite graph rank neither relies on nor replaces that structural argument.

5 Pillar IV: Algebraic Effects, Split-Phase Operational Handlers, and the Epistemic Effect Lattice

5.1 Split-Phase ITree Lowering versus the Asynchronous Handler Algebra

At the pure syntax and local reduction level (δ_v), factoring $\mathsf{Vis}(e : E(A), k : A \to X)$ into `emit` and `receive` is a syntactic bimorphism $\mathcal{S} : \mathsf{ITree}_{E,R} \to \mathsf{ITree}_{E_{\text{split}},R}$ over $E_{\text{split}} = \mathsf{Emit} \uplus \mathsf{Receive}$:

$$\mathcal{S}\big(\mathsf{Vis}(e,k)\big) \triangleq \mathsf{Vis}\Big(\mathsf{Emit}(e),\ \lambda id.\ \mathsf{Vis}\big(\mathsf{Receive}(id),\ \lambda a.\ \mathcal{S}(k(a))\big)\Big).$$

However, in Xia et al. [16], `interp` applies a monadic handler $h : E_{\text{split}} \rightsquigarrow \mathsf{StateT}(\mathcal{D}, M)$ synchronously inside the evaluator's thread, which cannot represent effects interrupted across the four distributed crash windows W_1 (post-reduce, pre-commit), W_2 (post-commit, pre-authorize vs. cancel), W_3 (post-authorize, in-flight transport), and W_4 (post-receipt, pre-settlement).

ALAN's mathematical contribution in Pillar IV is therefore the *Split-Phase Journaled Handler Algebra* ($\mathcal{H}_{\text{journal}}$), which interprets each committed intent $\iota \in \mathcal{D}.\mathsf{Int}$ as an asynchronous coalgebraic protocol evolving over a four-stage *consistent epistemic poset* $(\mathbb{O}_{\text{eff}}^{\text{cons}}, \sqsubseteq_{\text{info}})$:

$$\mathbb{O}_{\text{eff}}^{\text{cons}} \triangleq \{\bot_{\text{unauth}}\} \uplus \big\{\mathsf{Suppressed}(r) \mid r \in \mathsf{Reason}_{\text{deny}}\big\} \uplus \{\mathsf{Unknown}\} \uplus \big\{\mathsf{Known}(v) \mid v \in \mathsf{Val}\big\},$$

with covering relations $\bot_{\text{unauth}} \sqsubset_{\text{info}} \mathsf{Suppressed}(r)$ and $\bot_{\text{unauth}} \sqsubset_{\text{info}} \mathsf{Unknown} \sqsubset_{\text{info}} \mathsf{Known}(v)$. Crucially, $\mathsf{Suppressed}(r)$ and $\mathsf{Known}(v)$ are *strictly incomparable* in $\mathbb{O}_{\text{eff}}^{\text{cons}}$. Adjoining an improper top fault element $\top_{\text{conflict}}$ completes $\mathbb{O}_{\text{eff}}^{\text{cons}}$ into the five-kind bounded algebraic lattice $\widehat{\mathbb{O}}_{\text{eff}} = \mathbb{O}_{\text{eff}}^{\text{cons}} \uplus \{\top_{\text{conflict}}\}$, where $\mathsf{Suppressed}(r) \sqcup \mathsf{Known}(v) = \top_{\text{conflict}}$ (`corrupt("receipt without authorization")` at `external.go:496`) and $\mathsf{Known}(v_1) \sqcup \mathsf{Known}(v_2) = \top_{\text{conflict}}$ for $v_1 \neq v_2$ (`ErrConflict` at `external.go:322`).

Theorem 5.1 (The Cancellation–Authorization Dichotomy)**.** *For any committed intent $\iota \in \Gamma.\mathsf{Int}$ of run $x = (a, g)$ and concurrent closure $c \in \{\mathsf{CancelRun}(a, g, w), \mathsf{RetireSubject}(a, w)\}$:*

1. *If $c \prec \mathsf{Authorize}(\iota)$ in the store's serialization order, ι transitions to the maximal consistent state $\mathsf{Suppressed}(\texttt{source_closed})$ and non-execution of ι in $\mathcal{W}$ is guaranteed under trusted dispatch.*

2. *If $\mathsf{Authorize}(\iota) \prec c$, ι transitions to $\mathsf{Unknown}$ (refinable to $\mathsf{Known}(v)$ upon late receipt arrival from the pre-cancellation attempt), and no new mutating attempt for ι may ever be launched after c, even with an idempotency key.*

5.2 Separating Probabilistic Intelligence from Deterministic Execution

In `examples/evidence/evidence.go` and `examples/investigation/investigation.go`, LLM invocations (`evidence-model`) are compiled into split-phase `author.Request` and `author.Await` steps surrounded by pure deterministic guards: budget enforcement (`model_spent < model_limit`), tool-name verification (`plan_tool == "lookup"`), deterministic span-exclusion copying across revision rounds, and byte-exact citation provenance verification across `document`, `version`, `snippet`, `start`, and `end`.

6 Pillar V: Crash-Consistent Operational Semantics and the Two-Regime Fundamental Theorem

6.1 Two-Tier Configurations, the Recovery Operator, and the Window-W_3 Impossibility Theorem

We decompose a single-host ALAN configuration into $\Gamma = \langle \mathcal{D}, \mathcal{V}, \mathcal{W} \rangle$, where $\mathcal{D}$ is the durable SQLite WAL/FULL store, $\mathcal{V}$ is the volatile activation state, and $\mathcal{W}$ is the external world state. Let $\mathsf{Commit}(\langle \mathcal{D}, \mathcal{V}, \mathcal{W} \rangle) = \langle \mathcal{D}, \emptyset_{\mathcal{V}}, \mathcal{W} \rangle$ and $\mathsf{Crash}(\Gamma) \ni \langle \mathcal{D}, \emptyset_{\mathcal{V}}, \mathcal{W}' \rangle$. Upon recovery, $\mathsf{Recover}$ validates ledger integrity and increments activation epochs $e \mapsto e + 1$ for open runs. When an authorized non-idempotent external attempt or adapter attempt has no recorded receipt ($\iota \in \mathsf{Unresolved}_{\text{non-idem}}(\mathcal{D})$), ALAN *neither re-transmits the request nor fabricates a synthetic inbox observation* (`"unknown outcome has no result"` at `adapters.go:314`); instead, the waiting run remains quiescently suspended at `receive` while $\iota.ID$ is surfaced in `RunReport.UnknownExternal` / `UnknownAdapters` (`external_test.go:175--226`).

Proposition 6.1 (Impossibility of Unconditional Symmetric Crash Bisimulation in Non-Idempotent Window W_3)**.** *Let $\iota \in \mathsf{Unresolved}_{non\text{-}idem}(\mathcal{D})$ be an authorized in-flight intent in Window W_3 targeting a non-idempotent endpoint $\mathcal{W}_\ell^{non\text{-}idem}$ without an out-of-band read-only receipt-lookup channel, and let $k(v)$ be a continuation whose observable output depends on v. No deterministic recovery operator $\mathsf{Recover}(\langle \mathcal{D}, \emptyset_{\mathcal{V}}, \mathcal{W}' \rangle)$ can simultaneously guarantee (1) At-Most-Once World Safety ($\#\mathsf{Apply}_\ell \leq 1$ across $\mathcal{W}' \in \{\mathcal{W}, \mathsf{apply}(\mathcal{W}, \alpha)\}$) and (2) Unconditional Symmetric Forward Bisimulation with the uninterrupted execution where the live socket in $\mathcal{V}$ delivers v to $k(v)$.*

Proof. Because $\mathcal{D}$ is identical whether the request dropped before reaching $\mathcal{W}_\ell$ ($\mathcal{W}' = \mathcal{W}$) or mutated $\mathcal{W}_\ell$ just before the response socket died ($\mathcal{W}' = \mathsf{apply}(\mathcal{W}, \alpha)$), re-dispatching α violates (1) in the latter case ($\#\mathsf{Apply}_\ell = 2$), whereas withholding re-dispatch leaves k without v, violating (2) in the realization where the live socket returned v. □

6.2 The Two-Regime Fundamental Theorem of Durable Computation

Let $\approx_{\text{obs}}$ require logical store identity modulo activation epochs and symmetric future $\approx^{\Delta}_{I,\mathcal{B}}$ behavior. Identity and epoch-independent local reduction are necessary evidence, not an automatic proof: shared resource/closure interference, admissible response coupling and concrete scheduling remain obligations even with no unknown non-idempotent attempt.

Directed epistemic prefix/safety simulation $\preccurlyeq_{\text{ep}}$ requires historical identity at a matched committed cut and conservative information, no automatic retransmission of an unknown non-idempotent attempt, and explicit peer/reconciliation obligations. A peer's lack of an await dependency is insufficient for symmetric comparison: resource, authority, closure and shared-world interference must be absent or accounted for by a stable relation. While no later logical closure intervenes, affected runs await authentic evidence with the original unknown handle and reservation; no value is fabricated. Late evidence may advance $\mathsf{Unknown}$ to $\mathsf{Known}(v)$, but rejoining $\approx_{\text{obs}}$ also requires matched continuations, world, authority and residual resources. Closure may prevent consumption without invalidating retained evidence. Conservative safety alone does not establish the full simulation.

The supervisory public-output reconvergence condition (Markdown Definition 6.5) is application-specific, not a separately checkable substitution proof. Retry debits and root-correlated external actions remain visible. Hiding a private generation or `spent` register cannot supply exact-debit equivalence; "modulo retry debit" would need a separately defined observation relation and is not claimed here.

Theorem 6.2 (Two-Regime Durable Computation; Qualified Claim and Open Simulation Obligations)**.** *For a reachable configuration under a fixed semantic version:*

1. ***Part I (Historical Replay Completeness and Zero-Effect Soundness):*** *read-only* `Replay` *reconstructs historical projections and intents without dispatching effects or mutating the retained store.*

2. ***Part II (Proposed Symmetric Future Relation):*** *when* $\mathsf{Unresolved}_{non\text{-}idem}(\mathcal{D}) = \emptyset$*, recovery preserves the retained logical cut subject to the version/epoch protocol. The stronger* $\mathrm{Recover}(\Gamma_{crash}) \approx_{\mathrm{obs}} \mathsf{Commit}(\Gamma)$ *remains a proof obligation under matched admissible responses, authorized endpoint reconciliation, interference-stable resource/closure relations and an explicit schedule interpretation. Idempotence or receipt availability alone does not establish it.*

3. ***Part III (Conservative Unknown Safety and Proposed Directed Simulation):*** *recovery of an authorized non-idempotent/adapter attempt without a result retains its original uncertainty and spent reservation, fabricates no observation and does not automatically retransmit it. Full* $\mathrm{Recover}(\Gamma_{crash}) \preccurlyeq_{\mathrm{ep}} \Gamma$ *additionally requires the peer/reconciliation obligations above. No outer exact-debit bisimulation after retry follows from hiding; expenditure, effects and authority/generation observations must be accounted for.*

Replay argument and recovery limits. Part I's historical induction uses determinism of the reducer, immutable programs/observations and the read-only replay guard. For Part II, retaining projections/handles/charges and changing activation authority yields the same local successor only for the same selected observation and valid authority. Volatile ready-run position and in-flight workers are not retained semantic state. Idempotent retransmission requires current authorization and the endpoint contract; prior source authorization does not bypass final reservation/closure checks. Selected pure process cuts do not prove the general simulation.

For Part III, unknown-attempt eligibility and settlement rules support no-repeat/no-fabrication safety. Lemma 4.4 permits enabled peers to progress despite an unmatched receive, not identical peer histories under shared competition. Surviving live late transport and fresh-process unknown recovery are distinct environments. Neither late evidence nor abandonment/retry restores exact-debit equivalence merely by hiding. The general peer/reconciliation/retry simulation argument remains open.

Under an endpoint contract that deduplicates identical immutable keys and parameters, execution-scoped keys support at-most-once logical mutation within a generation across permitted physical retries. This endpoint-dependent safety fact is separate from the open future-simulation claim and does not authorize retransmission after closure.

Theorem 6.3 (Impossibility of Blind Cross-Generation Business Deduplication)**.** *Across a supervisor-initiated* `Retry` *from* $x_1 = (a, g)$ *to* $x_2 = (a, g+1)$*, the execution-scoped external keys* $\mathsf{ExtKey}(\mathcal{N}_{256}, (a, g), n, 0) \neq \mathsf{ExtKey}(\mathcal{N}_{256}, (a, g+1), n, 0)$ *are distinct. Consequently, computation recovery and outbox idempotence cannot prevent duplicate external mutation across generations unless the application threads a generation-invariant business key or reconciles* Unknown *prior to retry (witnessed by* `business:order-7` *in Phase 1C).*

7 Conclusion

ALAN demonstrates that the path beyond fragile agent orchestration is neither adding more prompt wrappers nor inventing privileged workflow node types. By uniting Felleisen's expressive minimality, Plotkin and Turi's mathematical operational semantics, Milner and Hewitt's concurrent topologies, Xia et al.'s Interaction Trees, and Schneider and Lamport's fault-tolerant state machines into a single crash-consistent calculus, ALAN makes agentic computation a rigorous, retainable, and composable mathematical object.

Qualification boundary. General congruence (Theorem 4.5) and dependent recovery simulation (Theorem 6.2) retain explicit proof obligations. Phase 1I checks bounded contract-relative guarantees, not equivalence of all conforming agents. Shared retry expenditure and external effects remain observable. The general loop-ranking argument and whole-dissertation formalization remain separate work; the bounded witnesses do not repair them.

References

[1] Gul Agha, Ian A. Mason, Scott Smith, and Carolyn Talcott. A foundation for actor computation. *Journal of Functional Programming*, 7(1):1–72, 1997.

[2] Mike Burrows. The Chubby lock service for loosely-coupled distributed systems. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*, pages 335–350, 2006.

[3] K. Mani Chandy and Leslie Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems (TOCS)*, 3(1):63–75, 1985.

[4] Alonzo Church. An unsolvable problem of elementary number theory. *American Journal of Mathematics*, 58(2):345–363, 1936.

[5] Matthias Felleisen. On the expressive power of programming languages. *Science of Computer Programming*, 17(1–3):35–75, 1991.

[6] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI '73)*, pages 235–245, 1973.

[7] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[8] Robin Milner, Joachim Parrow, and David Walker. A calculus of mobile processes, I and II. *Information and Computation*, 100(1):1–77, 1992.

[9] Marvin L. Minsky. Recursive unsolvability of Post's problem of "Tag" and other topics in theory of Turing machines. *Annals of Mathematics*, 74(3):437–455, 1961.

[10] Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.

[11] Gordon D. Plotkin. A structural approach to operational semantics. *Journal of Logic and Algebraic Programming*, 60–61:17–139, 2004. Originally published as Technical Report DAIMI FN-19, Aarhus University, 1981.

[12] Gordon D. Plotkin and Matija Pretnar. Handling algebraic effects. *Logical Methods in Computer Science*, 9(4:23):1–36, 2013. Conference version in ESOP 2009, LNCS 5502, pp. 80–94.

[13] Fred B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4):299–319, 1990.

[14] Daniele Turi and Gordon D. Plotkin. Towards a mathematical operational semantics. In *Proceedings of the 12th Annual IEEE Symposium on Logic in Computer Science (LICS '97)*, pages 280–291, 1997.

[15] Alan M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42(1):230–265, 1936.

[16] Li-yao Xia, Yannick Zakowski, Paul He, Chung-Kil Hur, Gregory Malecha, Benjamin C. Pierce, and Steve Zdancewic. Interaction trees: Representing recursive and impure programs in Coq. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–32, 2020.